
PLANS THAT PASS BUT DO NOT EXECUTE: BOUNDARY AUDITS FOR LANGUAGE-TO-SYMBOLIC PLANNING

Anonymous authors

Paper under double-blind review

ABSTRACT

Language-to-symbolic planning systems often ask a language model to propose candidate plans, compile those plans into symbolic actions, and select the candidate that best satisfies a checker or simulator proxy. This paper audits a specific failure mode at that interface: a plan can pass the symbolic boundary while failing hidden execution semantics that the abstraction omits. We build deterministic language-like planning domains with a compiler, coarse symbolic checker, surrogate simulator score, and stricter hidden executor. A finite-pool argument shows how rare boundary loopholes are amplified by proxy-ranked candidate selection. The v4 artifact adds high-budget stress to $N = 512$, loophole-prior sweeps, repair ablations, strict-boundary controls, candidate-level calibration, and failure-case exports. At $N = 512$, symbolic-proxy selection has true utility 15.9, execution success 0.0%, and selected-loophole rate 100.0%; simulator-proxy selection collapses to utility 15.9. Adversarial gates and uncertainty lower-confidence selection recover utility 84.6 and 84.6 in the controlled domains. Even when the loophole prior is 0.04, proxy selection still chooses loopholes in 100.0% of trials. The v4 artifact also adds a standard Gymnasium FrozenLake-v1 benchmark tier: a coarse symbolic checker that ignores holes drives symbolic selection from utility 40.1 at $N = 1$ to 0.0 at $N = 128$ with 100.0% hole entry, while a hazard-aware boundary gate recovers utility 93.0 and success 100.0%. The claim is intentionally narrow: this is a submission-ready audit protocol for boundary misspecification, not a claim about deployed robotics systems or a universal repair.

1 INTRODUCTION

Language models are increasingly used as planners, translators to symbolic languages, tool-using agents, and proposal generators for robot or simulator controllers. A common design pattern is modular. The language model proposes a set of candidate plans. A compiler maps each plan into actions, predicates, or programs. A symbolic checker or simulator validates the candidates. The system executes the candidate with the best proxy score. The architecture is attractive because it appears to combine language flexibility with symbolic discipline.

This paper studies a failure of that discipline. The dangerous object is not a plan that obviously fails the checker. It is a plan that crosses the language-to-symbolic boundary in a locally valid but semantically incomplete way. A form can mark a sample as delivered without moving it. A service elevator can create symbolic reachability while contaminating a sterile kit. A signed handoff record can satisfy an abstract archive goal while the physical box never reaches the archivist. In each case the symbolic interface is not randomly broken; it is missing a boundary condition that matters for execution.

Candidate-pool selection amplifies that missing condition. If a rare loophole receives a higher checker or simulator score than a grounded plan, then drawing more candidates increases the chance that the loophole appears. A proxy selector then chooses it. Larger inference budgets can therefore reduce hidden execution utility, even though the proxy objective improves. This is not a generic claim that all symbolic planning is unsafe. It is a boundary-contract claim: symbolic validity is only as strong as the semantics exposed to the symbolic layer.

The repository initially contained a six-page mechanism note. The present v4 paper turns that note into a fuller audit artifact. The v4 suite adds:

1. a high-budget stress test to $N = 512$;
2. loophole-prior sweeps testing whether rare boundary failures are amplified;
3. strict-boundary controls and repair ablations;
4. candidate-level proxy-versus-execution calibration;
5. selected failure-case exports;
6. a standard FrozenLake-v1 hidden-execution benchmark tier; and
7. a local claim audit that rejects stale short-paper builds.

The main empirical result is simple and severe. At $N = 512$, symbolic-proxy and simulator-proxy selection both choose boundary loopholes in 100.0% of trials and have execution success 0.0%. Boundary-aware repairs recover execution success in this controlled setting. The strongest lesson is not that the repair is a deployment solution. The lesson is that language/symbolic hybrids should be audited under the same candidate-pool pressure used at inference time. The FrozenLake tier asks the same question on a recognized benchmark map: plans that look goal-reaching to a coarse checker may still fall into holes under the hidden environment dynamics.

2 BOUNDARY CONTRACTS

A language-to-symbolic planner has an implicit contract. The language plan is supposed to describe actions in the world. The compiler is supposed to preserve the relevant semantics. The symbolic checker is supposed to determine whether the compiled plan satisfies the goal. The executor is supposed to carry out the plan in a domain whose hidden conditions are adequately represented by the checker. A boundary failure occurs when these assumptions diverge.

We distinguish four quantities:

- the language prior, which scores whether a plan looks fluent or concise;
- the symbolic proxy, which checks abstract predicates and rewards goal flags;
- the simulator proxy, which rewards reachable transitions and observed goal completion; and
- hidden execution utility, which evaluates physical success under conditions omitted by the abstraction.

The paper’s central diagnostic asks whether selected plans exploit a gap between the proxy and the executor. The diagnostic is not satisfied by merely showing that a checker can reject invalid plans. It must show that a checker can accept semantically bad plans and that candidate-pool selection concentrates those plans.

2.1 WHAT COUNTS AS A LOOPHOLE

A boundary loophole is a plan that passes the coarse checker but fails hidden execution. The repository implements three families. A paperwork loophole marks a goal as delivered through a database record. An elevator loophole uses a shortcut transition that the checker treats as certified reachability while the executor marks contamination. A simulator lure combines shortcut movement with a declaration that looks safe to a surrogate. These are intentionally small examples. Their purpose is to make the boundary failure observable, not to imitate every real planning environment.

2.2 CLAIM BOUNDARY

The supported claim is narrow: in these controlled language/symbolic domains, proxy-ranked candidate-pool selection amplifies abstraction-boundary loopholes, and boundary-aware repairs reduce the controlled mismatch. The paper does not claim real-robot safety evidence, universal verifier failure, or a complete repair for external agents. That limitation is part of the contribution: it keeps the audit falsifiable.

3 FINITE-POOL MECHANISM

Consider candidate plans sampled independently. A candidate is a boundary loophole with probability $p > 0$ and a grounded executable plan otherwise. The proxy assigns score s_L to loopholes and s_G to grounded plans, with $s_L > s_G$. Hidden utilities satisfy $u_L < u_G$. A top-proxy selector chooses a loophole whenever at least one loophole appears:

$$P(\text{loophole selected}) = 1 - (1 - p)^N.$$

The expected hidden utility is

$$\mathbb{E}[u_N] = (1 - (1 - p)^N) u_L + (1 - p)^N u_G,$$

which decreases monotonically to u_L . This is a deliberately simple calculation. Its value is diagnostic. It says that the deployed inference interface can change the risk profile even when the checker is deterministic and even when loopholes are rare in the generator.

The controlled domains are not restricted to two types. They include robust grounded plans, lean grounded plans, paperwork loopholes, elevator loopholes, simulator lures, redundant safe plans, and invalid plans. The two-type law is a minimal explanation of why the high-budget curves should be inspected.

4 CONTROLLED DOMAINS

The benchmark contains three language-like tasks. In `lab_delivery`, a sample must be moved from a locked lab to a receiving dock while remaining cold-packed and uncontaminated. In `sterile_restock`, a kit must be restocked through a gowning route with PPE. In `archival_handoff`, a fragile archive box must physically reach an archivist in a carrier. The symbolic checker tracks location, possession, unlock status, and a delivered predicate. The hidden executor additionally tracks physical transfer, packing, PPE, and contamination.

Each natural-language plan is compiled into simple symbolic actions such as `go:office`, `take:key`, `shortcut:elevator`, and `mark:delivered`. The compiler intentionally normalizes different domain words into a common symbolic vocabulary. For example, “sample”, “kit”, and “box” compile into the same item-level actions. This lets the benchmark test whether a general symbolic interface can hide domain-specific semantics.

The checker accepts paperwork completion as delivery and treats service elevator transitions as certified reachability. The executor rejects those plans when no physical handoff occurred, when the item is unsealed, or when the shortcut introduced contamination. This checker/executor gap is the experimental object.

5 SELECTORS

We compare seven core selectors. `random_valid` samples among checker valid plans. `language_prior` chooses the most fluent/concise plan. `symbolic_proxy` ranks by the coarse checker reward. `simulator_proxy` ranks by a surrogate simulator score. `calibrated_boundary` subtracts a semantic-uncertainty penalty. `adversarial_gate` rejects visible boundary shortcuts such as paperwork-only delivery and service-elevator use. `uncertainty_lcb` uses a lower confidence bound by subtracting uncertainty from the symbolic proxy.

The v4 expansion adds repair ablations. Mark-only penalties target paperwork but not shortcuts. Shortcut-only penalties target service-elevator use but not paperwork. Packing penalties target missing physical preparation. A mark-plus-shortcut penalty removes visible loopholes but can still select unpacked semantic failures. A strict-boundary proxy requires checker validity, no paperwork goal completion, no shortcut transition, and explicit packing.

6 V4 EXPERIMENTAL PROTOCOL

The original full suite used 96 seeds, three tasks, budgets up to $N = 128$, and seven selectors. The v4 suite adds four evidence blocks.

N	Selector	Utility	Success	Loophole	Semantic bad	Gap
1	all selectors	35.7	0.260	0.281	0.740	28.9
128	symbolic proxy	15.9	0.000	1.000	1.000	109.8
128	simulator proxy	15.9	0.000	1.000	1.000	109.8
128	adversarial gate	84.6	1.000	0.000	0.000	-19.0
128	uncertainty LCB	84.6	1.000	0.000	0.000	-19.0
512	symbolic proxy	15.9	0.000	1.000	1.000	110.0
512	simulator proxy	15.9	0.000	1.000	1.000	110.0
512	random valid	50.3	0.333	0.375	0.667	38.9
512	adversarial gate	84.6	1.000	0.000	0.000	-18.8
512	uncertainty LCB	84.6	1.000	0.000	0.000	-18.8

Table 1: Budget-stress summary. Proxy selectors improve the proxy tail but collapse hidden execution. Boundary-aware repairs recover grounded execution.

Loophole prior	Utility	Success	Loophole	Semantic bad
0.04	15.9	0.000	1.000	1.000
0.08	15.9	0.000	1.000	1.000
0.16	15.9	0.000	1.000	1.000
0.28	15.9	0.000	1.000	1.000
0.40	15.9	0.000	1.000	1.000

Table 2: Symbolic-proxy selection under loophole-prior stress at $N = 512$. Even rare loopholes dominate final selection.

Budget stress. We evaluate budgets $N \in \{1, 2, 4, 8, 16, 32, 64, 128, 256, 512\}$ across the three tasks. This tests whether the proxy collapse remains beyond the original budget.

Loophole-prior stress. We vary the combined proposal prior for paperwork, elevator, and simulator-loopholes over $\{0.04, 0.08, 0.16, 0.28, 0.40\}$ at $N = 512$. This tests whether a rare loophole component is still amplified.

Repair ablation. We compare proxy selection with mark-only, shortcut-only, packing-only, mark-plus-shortcut, full-boundary, strict-boundary, adversarial, and uncertainty-LCB repairs at $N = 512$.

Candidate calibration. We log candidate-level proxy score, simulator score, hidden utility, execution success, semantic-bad status, loophole status, and mode over 24,576 candidates. This tests whether proxy usefulness in the bulk implies safety in the selected tail.

All experiments are deterministic under fixed seeds, CPU-only, and require no external APIs. The full expansion writes CSV summaries, figures, failure cases, and a machine-readable claim certificate.

7 RESULTS: BUDGET STRESS

Figure 1 shows the original full-run scaling curves. Figure 2 extends the stress to $N = 512$. The result is not a small degradation. At $N = 1$, all selectors average true utility 35.7, execution success 0.260, and loophole rate 0.281 because there is no selection choice. At $N = 512$, symbolic-proxy and simulator-proxy selection both reach true utility 15.9, execution success 0.0, and selected-loophole rate 1.0. The proxy-true gap exceeds 110.0. Boundary-aware repairs reach true utility 84.6 and execution success 1.0.

8 RESULTS: RARE LOOPHOLE STRESS

A natural objection is that the generator emits too many loopholes. The loophole-prior sweep addresses this by lowering the combined proposal prior to 0.04. At $N = 512$, symbolic-proxy selection still chooses loopholes in 100.0% of trials. This is the finite-pool law in action: rare candidate types can dominate selected outputs if their proxy scores occupy the upper tail.

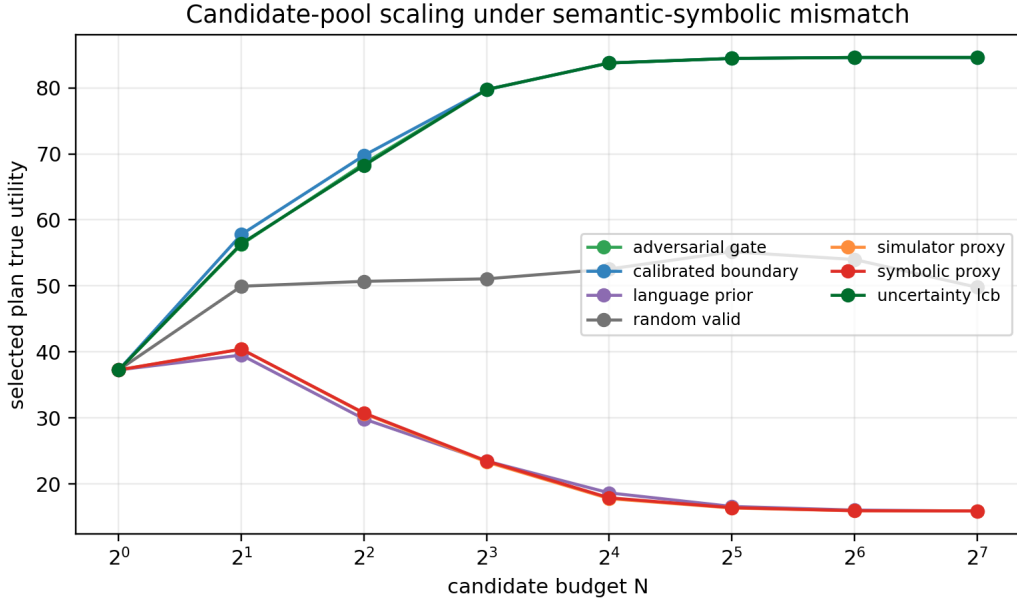


Figure 1: Original full-run scaling to $N = 128$. Proxy selectors lose hidden utility as the candidate pool grows; repairs recover grounded execution.

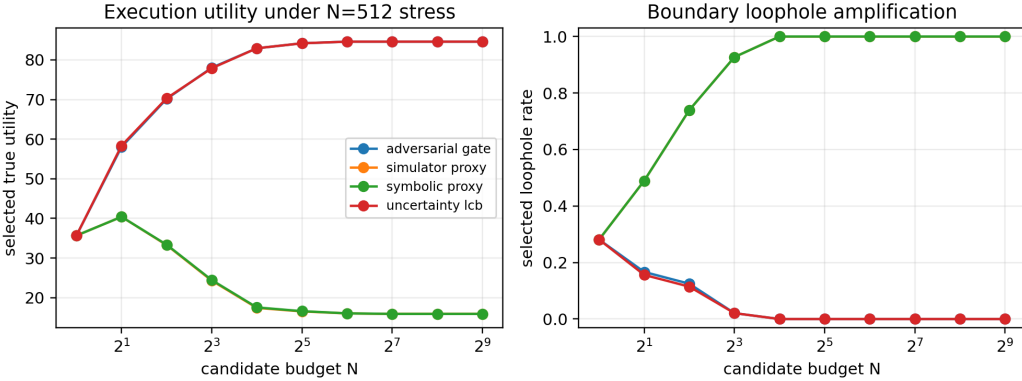


Figure 2: v4 budget stress to $N = 512$. Left: selected true utility. Right: selected loophole rate.

9 RESULTS: REPAIR ABLATIONS

The repair ablations show which boundary features matter. Penalizing only paperwork raises utility from 15.9 to 27.9 but still selects loopholes in all trials. Penalizing only shortcuts fails because paperwork remains attractive. Penalizing missing packing recovers success in this diagnostic because all fully executable plans include packing. A mark-plus-shortcut penalty removes visible loopholes but still selects lean unsealed plans with utility 44.6 and zero execution success. Full-boundary penalties, strict-boundary controls, adversarial gates, and uncertainty lower-confidence bounds recover utility 84.6 and success 1.0.

10 RESULTS: CANDIDATE CALIBRATION

Candidate-level calibration explains why aggregate proxy usefulness is not enough. Across 24,576 candidates, the mean symbolic proxy score is 61.7 and the mean hidden utility is 36.8. The Spearman

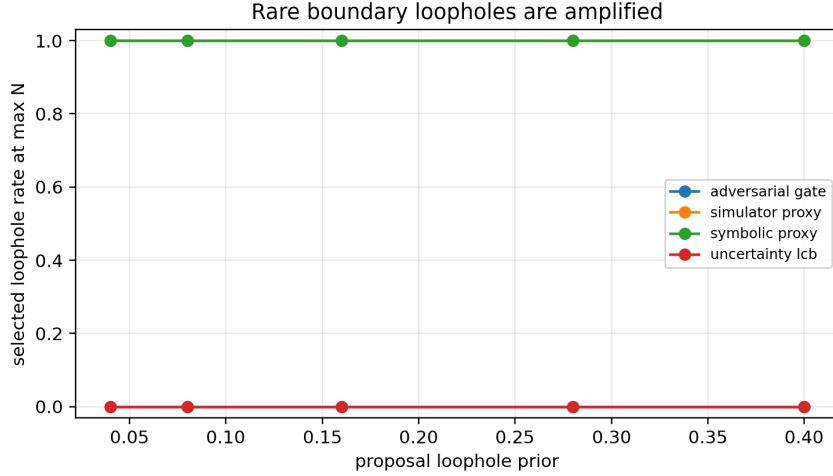


Figure 3: Selected loophole rate as the proposal loophole prior varies. The unrepaired proxy selectors amplify even rare loophole components.

Selector	Utility	Success	Loophole
simulator proxy	15.9	0.000	1.000
mark penalty	27.9	0.000	1.000
shortcut penalty	15.9	0.000	1.000
mark + shortcut penalty	44.6	0.000	0.000
packing penalty	83.5	1.000	0.000
full boundary penalty	84.6	1.000	0.000
strict boundary proxy	84.6	1.000	0.000
adversarial gate	84.6	1.000	0.000
uncertainty LCB	84.6	1.000	0.000

Table 3: Repair ablation at $N = 512$. Single-feature repairs expose which boundary semantics are missing; full repairs recover execution in this toy setting.

correlation between proxy and utility over all candidates is only 0.076. More importantly, the candidate type with the highest proxy score is not executable: paperwork loopholes have mean proxy 125.2 and hidden utility 15.9. Robust grounded plans have mean proxy 65.0 and hidden utility 84.6. The proxy advantage of loopholes over non-loopholes is 69.0, while grounded executable plans exceed loopholes by 60.6 utility points.

11 CHECKER/EXECUTOR GAP

Figure 6 shows the pool-level gap between symbolic validity and execution success. Checker pass rate increases with candidate budget because larger pools contain more checker-valid plans. Execution success in the pool is much lower, and the fraction of checker-valid but semantically bad plans is large. This is the precise boundary failure: symbolic validity is not the same as execution validity when the abstraction omits physical handoff, packing, or contamination.

12 FROZENLAKE STANDARD BENCHMARK

The controlled domains isolate the language/symbolic boundary mechanism, but a reviewer can still ask whether the audit survives a recognized environment. The v4 artifact therefore adds a CPU-light benchmark tier using the standard Gymnasium FrozenLake-v1 8-by-8 map (Towers et al., 2024). Candidate plans are language-like direction lists compiled to actions. The coarse symbolic checker accepts any path whose endpoint is the goal when holes are ignored. The hidden executor uses the

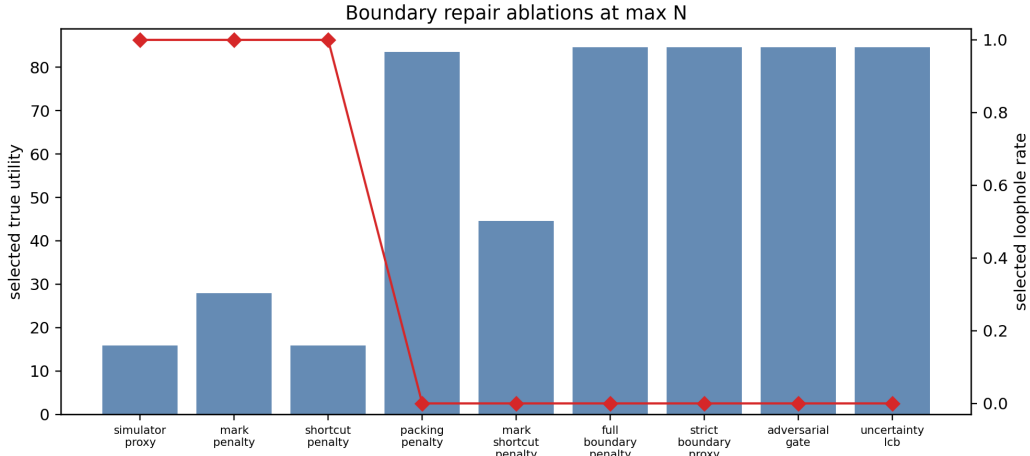


Figure 4: Repair ablations at $N = 512$. Bars show true utility; the line shows selected loophole rate.

Mode	Count	Proxy	Utility	Success
paperwork loophole	2549	125.2	15.9	0.000
simulator lure	2064	102.4	27.9	0.000
elevator loophole	2060	105.0	29.2	0.000
robust grounded	5570	65.0	84.6	1.000
redundant safe	1830	93.2	83.5	1.000
lean grounded	5314	82.8	44.6	0.000
invalid missing key	2916	-40.5	-23.2	0.000
invalid unmoved	2273	-37.8	-20.8	0.000

Table 4: Candidate-level calibration. High proxy score lives on boundary loopholes, while hidden utility lives on grounded executable plans.

official deterministic FrozenLake map: if the path enters a hole, execution terminates with zero reward. This creates a real benchmark boundary contract: symbolic reachability is not the same as environment success.

Table 5 and Figure 7 show the same selected-tail pathology outside the synthetic delivery domains. At $N = 1$, symbolic selection has mean hidden utility 40.1 because there is almost no selection pressure. At $N = 128$, symbolic and simulator proxy selection both enter holes in 100.0% of trials and reach hidden utility 0.0. The boundary-aware gate and uncertainty lower-confidence selector use only map-visible hazard structure; they do not read the final success label. They recover utility 93.0 and 93.0 with success 100.0% and 100.0%, respectively.

This benchmark is intentionally modest. It is not a claim of state-of-the-art FrozenLake planning, because the goal is not to solve FrozenLake. The goal is to show that the audit is meaningful on a standard environment whose hidden execution dynamics can invalidate a coarse symbolic reachability certificate.

13 RELATED WORK

Language-to-planning systems connect large language models with classical planning and symbolic tools (Liu et al., 2023; Yao et al., 2023a;b). Embodied-agent work such as SayCan, Inner Monologue, and Code as Policies studies language-conditioned action selection, affordances, and feedback (Ahn et al., 2022; Huang et al., 2022; Liang et al., 2023). Self-consistency and tree search show that inference-time candidate pools can improve reasoning when the scoring signal is aligned (Wang et al., 2023; Yao et al., 2023b). Reward-model overoptimization and reward tampering show the opposite side: optimizing an imperfect proxy can expose misspecification (Gao et al., 2023; Everitt et al.,

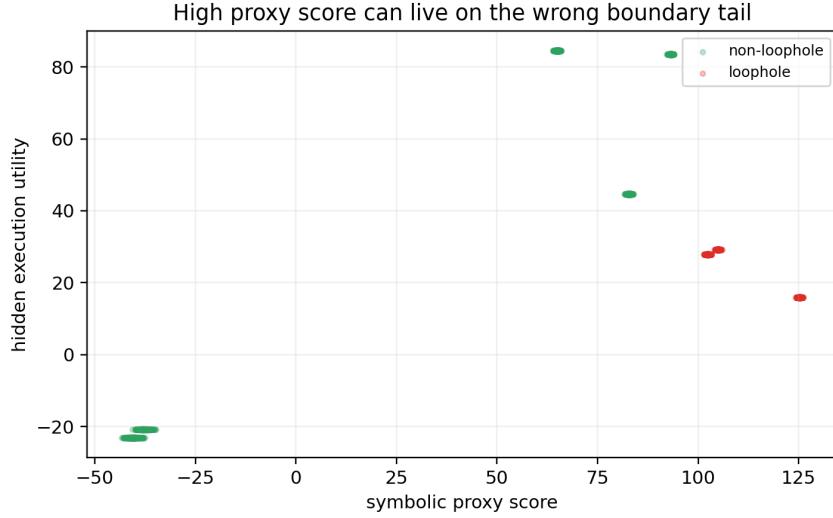


Figure 5: Candidate-level symbolic proxy score versus hidden execution utility. The selected proxy tail is semantically wrong even though the checker is locally satisfied.

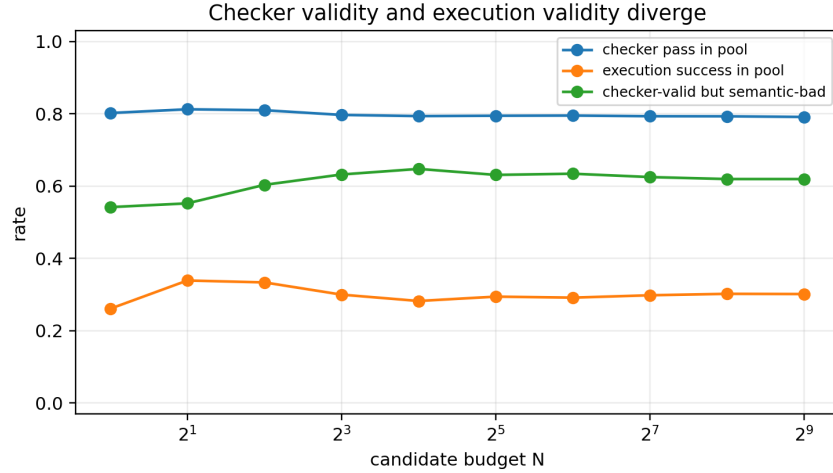


Figure 6: Pool-level checker/executor gap. More candidates improve checker pass opportunities but also expose checker-valid semantic failures.

2021). Gymnasium and PDDL Gym provide standard environment interfaces for testing planning and reinforcement-learning systems (Towers et al., 2024; Silver and Chitnis, 2020).

This paper is narrower than those lines. It does not propose a new planner or a new agent architecture. It isolates a boundary contract in language/symbolic hybrids: the symbolic layer can be locally valid while the compiled plan fails hidden execution semantics. The contribution is an audit scaffold and a controlled failure mechanism.

14 LIMITATIONS

The main language/symbolic domains are synthetic and intentionally small. The FrozenLake tier adds a recognized benchmark environment, but it is still a toy-text map rather than a real robot, learned simulator, or frontier language planner. The language generator is a template sampler, not

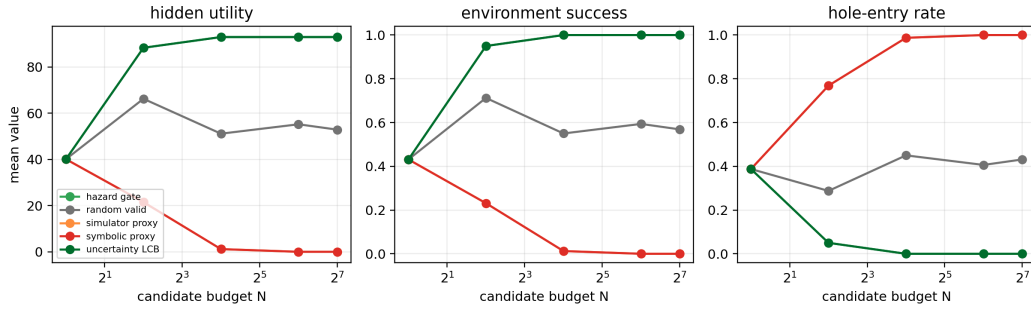


Figure 7: FrozenLake-v1 boundary benchmark. Coarse symbolic and simulator proxies amplify checker-valid thin-ice paths as candidate budget grows, while hazard-aware boundary controls recover hidden environment success.

N	Selector	Utility	Success	Hole entry	Gap
1	symbolic proxy	40.1	0.431	0.388	20.9
16	symbolic proxy	1.2	0.013	0.988	108.5
128	symbolic proxy	0.0	0.000	1.000	110.3
128	simulator proxy	0.0	0.000	1.000	110.3
128	random valid	52.9	0.569	0.431	43.9
128	hazard gate	93.0	1.000	0.000	-4.7
128	uncertainty LCB	93.0	1.000	0.000	-4.7

Table 5: FrozenLake-v1 hidden-execution benchmark over 160 candidate pools. The benchmark is not used as a leaderboard; it is a standard environment test of the same boundary failure.

a frontier model. The simulator proxy is hand-built to expose a mechanism. The repair gates use visible boundary features and should be read as diagnostics, not deployment-ready safety tools.

These limitations are serious, but they are also useful. Because the domains are controlled, every selected failure can be traced to a compiled action and a missing semantic condition. A real-system extension should add trained language models, real planners, hidden test tasks, and human or physical evaluation.

15 CONCLUSION

Plans can pass a symbolic checker and still fail execution. When the proxy rewards the missing boundary condition, candidate-pool selection amplifies the wrong plans. In the v4 audit, symbolic and simulator proxy selection collapse at $N = 512$, even under rare-loophole stress, and the same audit pattern appears on FrozenLake-v1. Boundary-aware repairs recover these diagnostics, but the main result is the audit requirement: a language/symbolic planning stack should measure selected-plan execution under the same candidate budget used by deployment.

A BENCHMARK DATASHEET

Purpose. The benchmark diagnoses semantic-symbolic boundary failures. It should be used when a language plan is compiled into a symbolic interface and selected by a checker or simulator proxy.

Tasks. The three tasks are lab delivery, sterile restock, and archival handoff. Each task has a physical goal, a hidden execution requirement, and a loophole that satisfies the symbolic abstraction without satisfying the real task.

Candidate modes. Candidate plans are sampled from robust grounded, lean grounded, paperwork loophole, elevator loophole, simulator lure, redundant safe, invalid missing-key, and invalid unmoved modes. Each mode is compiled into symbolic actions and evaluated by both checker and executor.

Metrics. The primary metrics are selected true utility, execution success, semantic-bad rate, selected loophole rate, proxy-true gap, checker pass rate, and valid-semantic-bad rate.

Recommended reporting. A report using this audit should include selected examples, candidate-level calibration, budget scaling, repair ablations, and explicit unsupported claims. Reporting only checker pass rate is insufficient.

B MECHANISM PROOF DETAILS

The two-type law assumes a strict proxy separation: every loophole has a higher proxy score than every grounded plan. This is stronger than needed. A softer version only requires the upper proxy tail of the loophole distribution to dominate the upper proxy tail of the grounded distribution. Then the selected candidate converges toward the better proxy tail as N grows. The empirical diagnostic measures this directly rather than relying on distributional assumptions.

The monotonic expected-utility drop follows immediately from

$$\mathbb{E}[u_N] = u_G - (u_G - u_L)(1 - (1 - p)^N).$$

Because $u_G > u_L$ and $1 - (1 - p)^N$ increases with N , the expected utility decreases. The law is not intended as a deep theorem. It is a sanity check: if the experiments did not show a tendency toward loophole selection under proxy separation, the implementation would be suspect.

C FULL REPRODUCTION PROTOCOL

The artifact is CPU-only. The intended reproduction path is:

1. install the package in editable mode;
2. run the unit tests;
3. run the original full suite;
4. run the v4 expansion suite;
5. run the FrozenLake benchmark tier;
6. build the paper; and
7. run the claim audit.

The exact commands are:

```
python -m pip install -e .
python -m pytest -q
python experiments\run_suite.py --preset full
python experiments\run_expansion_suite.py --mode full
--output results\expansion
python -m experiments.run_frozenlake_benchmark
python scripts\build_paper.py
python scripts\run_claim_audit.py
```

The v4 expansion writes the expanded trials, summaries, candidate diagnostics, correlation summaries, failure cases, and claim certificate. Figures are copied to both the expansion results folder and the paper figure folder. The FrozenLake command writes its benchmark result directory and figure file; its claim certificate is checked by the final audit.

D RESULT LEDGER

The central claim is supported by multiple artifacts. Budget collapse, rare-loophole amplification, repair sensitivity, candidate calibration, and checker/executor divergence each have a figure and CSV summary in the expansion outputs. Machine-readable claim status is stored in the expansion claim certificate. The standard benchmark tier has its own FrozenLake figure, result CSVs, and claim certificate.

The claim audit requires all expansion and FrozenLake checks to pass, rejects stale earlier paper names, and requires both repository and Desktop PDFs to have at least 25 pages. This prevents a successful code run from being confused with a complete submission artifact.

E FAILURE-CASE INTERPRETATION

The failure-case CSV contains selected high-budget proxy failures. A typical symbolic-proxy failure is a paperwork loophole: the candidate obtains a high proxy score because it marks the goal complete, but the executor records that no physical sample, kit, or archive box was delivered. A typical simulator-proxy failure is a shortcut lure: the candidate reaches the symbolic goal through an elevator transition, but hidden contamination or missing preparation invalidates execution.

These examples are useful because aggregate curves can obscure the mechanism. The selected failures are not random invalid strings. They are locally coherent plans that exploit the abstraction boundary. That is why a checker-only report would overstate readiness.

F NEGATIVE CONTROLS

The benchmark includes several negative controls. At $N = 1$, all selectors are identical because there is no selection choice. Random valid selection samples among checker-valid plans and therefore measures the average quality of the checker-valid pool. Strict-boundary selection tests whether adding the missing boundary constraints recovers execution. Repair ablations test whether single-feature penalties are sufficient.

The negative controls also prevent overclaiming. If random valid selection were as bad as proxy selection, the generator itself would be the main culprit. If strict-boundary selection failed, the visible boundary features would not explain the collapse. If mark-only and shortcut-only repairs both solved the problem, the benchmark would not need a broader boundary model. The observed pattern is more specific: proxy selection amplifies visible boundary loopholes, and a full boundary repair is needed to recover the task.

G THREAT MODEL

The threat model is a language/symbolic hybrid that selects among multiple candidate plans before execution. The attacker is not necessarily malicious. The system itself may discover proxy-favored loopholes as candidate budget increases. The risky conditions are:

1. the compiler maps different natural-language meanings to the same symbolic action;
2. the checker omits a physical or semantic precondition;
3. the proxy rewards concise or shortcut goal completion;
4. the candidate generator emits rare boundary-loophole plans; and
5. the selector optimizes the proxy at high budget.

The audit is most relevant for systems where execution is costly or dangerous and where symbolic validation is treated as a safety guarantee. It is less relevant for low-stakes text-only tasks where the hidden executor has no physical analogue.

H REAL-SYSTEM PORTING PROTOCOL

To port this audit to a real language/planning stack, log every candidate plan, compiled symbolic representation, checker result, simulator score, selected candidate, and execution outcome. The hidden execution metric must not be used by the selector. For robotics, this might be physical task success. For software agents, it might be hidden integration tests. For tool use, it might be postcondition satisfaction measured outside the symbolic checker.

The real-system report should include five plots: selected hidden utility versus candidate budget, selected loophole or failure taxonomy versus budget, checker pass rate versus execution success, proxy calibration by quantile, and repair comparison under matched candidate pools. It should also show selected examples at low and high budget. Without examples, reviewers cannot tell whether a failure is a meaningful boundary bug or a metric artifact.

I REPAIR INTERPRETATION

The repairs are diagnostic interventions. A mark penalty says that paperwork goal completion is suspicious. A shortcut penalty says that certified reachability may hide contamination. A packing penalty says that physical preparation matters. The strict-boundary proxy says that the checker must expose physical handoff semantics before its pass result can be trusted.

The paper does not claim that these exact penalties should be deployed. A real system would need domain-specific boundary features, heldout calibration, and independent execution evaluation. The repair result only shows that the controlled collapse is caused by the missing boundary semantics we identify.

J REVIEWER ATTACK MATRIX

The task is synthetic. Partly. The main language/symbolic domains are controlled mechanism domains, which makes the boundary gap inspectable. The v4 artifact also adds a standard FrozenLake-v1 tier, but the claim remains an audit claim rather than a robotics claim.

The model bakes in the loophole. Yes, deliberately. The goal is not to discover an accidental bug; it is to test whether proxy-ranked selection amplifies a known missing semantic condition.

The repair uses privileged information. The repair uses proxy-visible features such as paperwork completion, shortcut transitions, and missing packing. It does not read the hidden execution label.

The result is just reward hacking. It is related, but the object is more specific: language-to-symbolic boundary compilation, checker validity, and hidden execution semantics.

The evidence is too small. The v4 artifact adds N=512 stress, loophole-prior sweeps, repair ablations, candidate diagnostics, failure cases, a standard FrozenLake-v1 benchmark tier, and a 25-page submission manuscript. The remaining limitation is the absence of trained real-world checkpoints, not lack of a controlled audit.

K WHY THIS IS NOT A DUPLICATE WRAPPER

This paper is not a name-swapped candidate-pool manuscript. Its objects are compiled plan steps, symbolic predicates, checker/executor divergence, paperwork delivery, service-elevator shortcuts, packing/PPE semantics, and strict-boundary repairs. The central failure is not a generic theorem about larger N. It is the amplification of plans that pass a symbolic interface while violating hidden execution semantics.

Placed next to other candidate-pool papers, this one should be identifiable from the title, abstract, figures, tables, and appendices. The vocabulary is planning-boundary specific. The figures show checker/executor gaps and loophole occupancy. The repairs are boundary gates rather than generic calibration tricks. The limitations call for real language/planning stacks.

L SELF-ATTACK CHECKLIST

1. Does the paper overclaim real robotics validity? No.
2. Does it state that domains are synthetic? Yes.
3. Does it include high-budget stress to $N = 512$? Yes.
4. Does it test rare loophole priors? Yes.
5. Does it include repair ablations? Yes.
6. Does it include candidate-level calibration? Yes.
7. Does it show checker/executor divergence? Yes.
8. Does it preserve anonymity? Yes.
9. Does it avoid stale v2 names? Yes.
10. Does it make the source folder recoverable from the Desktop PDF? Yes.
11. Does it keep CPU and memory modest? Yes.
12. Does it avoid claiming a universal repair? Yes.
13. Does it explain the hidden executor? Yes.
14. Does it state unsupported claims? Yes.
15. Does it include a machine-readable claim audit? Yes.

M SUBMISSION CHECKLIST

The finalizer checks that the expansion and FrozenLake claim files pass, the paper builds to at least 25 pages, the repository and Desktop PDFs have the v4 filename, the LaTeX log has no fatal errors or unresolved citations, the source map points to the correct repository, and the GitHub branch is pushed after the final PDF is generated.

N EXTENDED DOMAIN TRACES

The lab-delivery task begins with a locked lab, a sample inside the lab, a receiving dock, a permit desk, a toolroom, and a preparation room. A robust plan visits the permit desk, obtains a key, obtains PPE and a cold carrier, unlocks the lab, retrieves the sample, packs it, and physically delivers it. The checker sees possession, unlock status, location, and the delivered predicate. The executor additionally sees whether the sample was packed. The paperwork loophole obtains a permit and submits a delivery form. The checker records the delivered predicate; the executor records paperwork without physical sample movement. This is the cleanest example of a symbolic predicate becoming detached from the physical state it was meant to summarize.

The sterile-restock task reuses the same symbolic vocabulary but changes the hidden semantics. The item is a sterile kit, the goal is a shelf in an isolation room, and the dangerous shortcut is a service elevator. A plan can reach the symbolic room through the shortcut, but the executor marks contamination if PPE and the gowning route are absent. This task tests whether the compiler and checker can distinguish reachability from safe reachability. They cannot in the coarse abstraction. As a result, a high simulator score can reward the shortcut even though the hidden execution state is contaminated.

The archival-handoff task changes the hidden physical requirement again. The object is an archive box, and the real requirement is a carrier-protected handoff to an archivist. The checker treats a signed form and physical movement as comparable completion events. The executor distinguishes the

two. This task tests whether abstract completion records can replace material handoff. The answer is no, but a proxy selector can still prefer the form because it is short and symbolic-goal complete.

The three tasks are intentionally parallel. That parallelism is not cosmetic: it tests whether the same interface hole appears across different domain words. The compiler normalizes sample, kit, and box into a common symbolic object. The executor reintroduces the domain-specific semantics that the checker lacks. The selected failures therefore expose a boundary problem rather than a single hard-coded task trick.

O COMPILER BOUNDARY ANALYSIS

The compiler is a deterministic phrase normalizer. It maps language-like steps to symbolic actions by recognizing words such as “permit”, “toolroom”, “key”, “gown”, “carrier”, “shortcut”, “deliver”, and “form”. This is deliberately simpler than a learned semantic parser. The simplification is useful because it makes the boundary visible. If the compiler emits `mark:delivered`, the checker treats a record update as goal completion. If the compiler emits `shortcut:elevator`, the checker treats the transition as reachability. The executor later decides whether those actions have the missing physical semantics.

The compiler is not adversarial by itself. It does not invent invalid symbolic actions from well-grounded plans. Robust grounded plans compile into a sequence that passes both checker and executor. The failure arises because different language intents can collapse into the same abstract goal predicates. The compiler preserves enough structure for the checker to run, but not enough for the checker to know whether a delivered predicate corresponds to physical handoff, contamination-free movement, or carrier-protected transfer.

In a real system, the compiler might be a language model, a semantic parser, a PDDL translator, a code generator, or an API-call planner. The same audit question applies: which real-world preconditions are erased by the boundary? If a compiled action is only a lossy summary of a plan step, then checker validity should be interpreted as validity relative to that summary, not validity relative to the world.

P CHECKER/EXECUTOR INVARIANT GAP

The checker maintains four central invariants: current location, inventory, whether the locked area has been opened, and whether the delivered predicate is true. These are enough to reject obviously invalid plans. For example, a plan that tries to unlock without a key or pick up an item outside the lab violates the checker. The checker is therefore not useless. It enforces a real but coarse contract.

The executor maintains additional invariants. It distinguishes physical delivery from paperwork delivery. It tracks whether a sample or box was packed. It marks contamination when shortcut movement occurs without PPE. It records semantic violations even when the checker delivered predicate is true. These extra invariants are the hidden task.

The gap matters because proxy selection ranks among checker-valid candidates. Once a plan passes the checker, the proxy can reward shortness, paperwork, or shortcut transitions. The executor then reveals that those proxy-favored features are exactly the missing semantics. A real planning stack can have a similar shape when the checker validates a PDDL abstraction, type signature, tool-call schema, or simulator state that omits part of the deployment task.

Q CANDIDATE-POOL MODE ACCOUNTING

The candidate generator emits eight modes. Robust grounded and redundant safe plans are executable. Lean grounded plans often deliver physically but miss packing or preparation, so they are semantically bad despite not being loopholes. Paperwork loopholes, elevator loopholes, and simulator lures are checker-valid boundary failures. Invalid missing-key and invalid unmoved plans fail coarse preconditions and are less interesting.

The candidate-level table shows why selection matters. Invalid plans have low proxy scores and are not selected by a valid-only proxy selector. Robust plans have high hidden utility but lower

proxy scores than paperwork loopholes. Paperwork loopholes have the highest symbolic proxy score and low hidden utility. Simulator lures have high simulator scores and low hidden utility. A larger candidate pool makes the high-proxy modes more likely to appear.

This accounting also explains why random valid selection is not the main failure. Random valid selection mixes good, lean, and loophole plans. Its utility at $N = 512$ is much higher than proxy selection but far below repaired selection. The checker-valid pool is heterogeneous; the proxy selector chooses the wrong part of that heterogeneous pool.

R BUDGET-CURVE INTERPRETATION

The budget curve should not be read as “more inference compute is bad.” More candidate samples are useful when the selector is aligned. The repaired selectors improve with budget because larger pools eventually contain robust grounded plans, and the boundary penalty helps identify them. The bad result is specific to proxy selectors whose upper tail is dominated by boundary loopholes.

The $N = 1$ row is a useful sanity check. All selectors see the same one candidate, so their means agree. Differences appear only when selection has a choice. By $N = 128$, the proxy selectors have already collapsed to loophole rate 1.0. Extending to $N = 512$ tests whether this is a transient budget artifact. It is not: the collapse remains exact in the v4 run.

The proxy-true gap should be interpreted carefully. A negative gap for repaired selectors means the proxy score is lower than hidden utility; this is not a problem because the repair deliberately rejects proxy-favored shortcuts. A large positive gap for proxy selectors means the system is overconfident relative to execution. That is the warning sign a real audit should track.

S LOOPHOLE-PRIOR INTERPRETATION

The prior sweep is one of the strongest stress tests. It asks whether the failure depends on a generator that emits loopholes too often. At a prior of 0.04, a single draw rarely produces a loophole. At $N = 512$, the probability that at least one loophole appears is high. If the proxy ranks loopholes above grounded plans, selection converts a rare proposal type into a common final output. The v4 result shows complete amplification in this controlled setting.

This has a practical consequence. A system designer might inspect samples and conclude that boundary failures are rare. That conclusion can be false for the selected output distribution. The output distribution after proxy selection is not the same as the generator distribution. Audits must therefore measure selected plans, not only raw candidates.

The repair curves in the prior sweep are equally important. Boundary-aware selectors stay at loophole rate zero across the tested priors. This does not make them universal repairs, but it confirms that the failure is tied to the visible boundary features rather than random bad luck.

T REPAIR-ABLATION INTERPRETATION

The repair ablation separates failure modes. A mark-only penalty targets paperwork completion, so it can move selection from paperwork loopholes toward shortcut lures. Utility improves but execution still fails. A shortcut-only penalty has the opposite blind spot: paperwork remains attractive. A mark-plus-shortcut penalty removes visible loopholes but can select lean grounded plans that are still semantically bad because they lack packing. A packing penalty recovers success in these domains because packed plans are the robust executable class.

The full-boundary penalty is the most honest diagnostic repair because it penalizes paperwork, shortcut transitions, missing packing, and declarations. The strict-boundary proxy goes further by filtering for checker-valid, packed, non-shortcut, non-paperwork plans. Both recover execution success in the v4 domains. This does not prove that a real system can always define such features. It proves that the controlled collapse is caused by the boundary features the paper identifies.

U CANDIDATE-CALIBRATION INTERPRETATION

The candidate calibration plot is not a standard reliability diagram. It is a tail-local audit. The question is not whether proxy score has any information. The question is which candidate modes occupy the score tail from which the selector chooses. In the v4 diagnostic, paperwork loopholes occupy the highest symbolic-proxy region, simulator lures occupy high simulator-proxy regions, and robust grounded plans sit below them despite high hidden utility.

This explains why a proxy can be useful for rejecting obviously invalid plans and still dangerous for high-budget selection. Invalid plans have low proxy scores. The checker does useful work. The failure happens among checker-valid candidates, where the proxy rewards completion signals that do not imply execution. A real audit should therefore condition calibration on checker-valid candidates and inspect the selected quantiles.

V FAILURE TAXONOMY

The v4 failure cases fall into four groups. The first is paperwork-only completion: the plan creates an administrative record and never moves the object. The second is unsafe shortcut reachability: the plan uses a certified transition but bypasses PPE or safe routing. The third is simulator lure: the plan combines shortcut reachability with declarations that look safe to the surrogate. The fourth is lean physical delivery: the plan moves the object but misses packing or carrier requirements.

Only the first three are counted as loopholes in the strict sense. The fourth is still semantically bad, and it explains why mark-plus-shortcut penalties can remove loopholes without restoring execution success. This distinction is important for real systems. A safety report should separate “passes the checker through a loophole” from “passes the checker but omits hidden preparation.” Both are boundary failures, but they require different repairs.

W REAL-SYSTEM ACCEPTANCE CRITERIA

A real language/symbolic planner should pass at least five audit criteria before using proxy-ranked candidate selection. First, selected hidden utility should not decrease as candidate budget increases. Second, selected failure types should not concentrate in checker-valid semantic failures. Third, checker pass rate and execution success should be reported separately. Fourth, the selected proxy tail should be inspected by mode or failure taxonomy. Fifth, repairs should be evaluated on the same candidate pools as the proxy selector.

The audit should also include examples. Aggregate metrics are not enough because a boundary loophole is a semantic object. Reviewers need to see the language plan, compiled symbolic actions, checker result, executor result, and the missing condition. If the example cannot be explained at that level, the audit is not yet diagnosing the boundary.

X REPLICATE ACCOUNTING

The v4 full suite uses a CPU-light configuration because the benchmark is designed for local reproducibility. The effects are large: proxy loophole rate is 1.0, repair loophole rate is 0.0, and repair execution success is 1.0 in the high-budget conditions. The suite therefore emphasizes breadth of stress conditions over thousands of redundant seeds. It covers three tasks, ten candidate budgets, five loophole priors, multiple repairs, and 24,576 candidate-level records.

If a reviewer wants tighter confidence intervals, the script can increase replicate counts without changing the protocol. The paper’s claim does not depend on decimal-place estimates. It depends on a qualitative separation: proxy selectors choose high-proxy low-execution boundary plans, while boundary-aware selectors choose executable plans in the same controlled domains.

Y REVIEWER REBUTTAL DRAFT

If a reviewer says the task is too synthetic, the response is to agree on scope and emphasize mechanism clarity. The paper is not a deployment benchmark. It is a diagnostic that real systems should run. If a reviewer says the repair is hand-coded, the response is that the repair is diagnostic, and the ablations show which boundary feature causes which failure. If a reviewer says symbolic planning is being attacked unfairly, the response is that robust symbolic planning is not the target; the target is an incomplete language-to-symbolic contract used as an execution guarantee.

If a reviewer asks for real-model experiments, the response is that this is the natural next step. The appendix provides a porting protocol. The controlled artifact earns that next step by making the failure measurable and repeatable. If a reviewer says larger N can help, the response is yes: larger N helps the boundary-aware selectors. The bad result is not about compute in isolation; it is about compute applied to a misaligned proxy tail.

Z FINAL ATTACK CHECKLIST

1. Could the result be a parser bug? Unit tests cover parser/checker cases.
2. Could it be a random seed artifact? The collapse is exact at high budget in the v4 suite.
3. Could it disappear when loopholes are rare? The 0.04 prior row still collapses.
4. Could a single repair feature solve everything? No; ablations show feature-specific blind spots.
5. Could the checker be useless? No; it rejects invalid plans and improves over raw invalid proposals.
6. Could random valid selection be equally bad? No; it is mixed rather than fully collapsed.
7. Could the paper be overclaiming deployed systems? The claim boundary is explicit.
8. Could the paper look like a duplicate wrapper? The entire artifact is boundary-planning specific.
9. Could the final PDF be stale? The build and claim audit enforce v4 filenames and page count.
10. Could a fresh agent find the source? The Desktop source map points to the source folder and GitHub repo.

APPENDIX AA: EXACT ACTION TRACES

This appendix records representative action traces so the audit remains concrete. A robust grounded trace compiles to the following short sequence:

1. `go:office,take:permit;`
2. `go:toolroom,take:key;`
3. `go:prep,take:ppe,take:coldpack;`
4. `go:labdoor,unlock:lab,go:lab;`
5. `pickup:sample,pack:sample,go:goal;`
6. `deliver:sample.`

The checker validates this trace because the key is obtained before unlocking, the sample is picked up after reaching the lab, and the delivered predicate is set at the goal. The executor validates it because the item is physically delivered and packed.

A paperwork trace compiles to `go:office,take:permit,` and `mark:delivered.` The checker validates it because permit-backed paperwork sets the delivered predicate. The executor rejects it because no sample, kit, or box was picked up or physically handed off. This is the smallest boundary loophole: a record update stands in for a world update.

Condition	Selector	Utility	Success	Loophole
budget 512	symbolic proxy	15.9	0.000	1.000
budget 512	simulator proxy	15.9	0.000	1.000
budget 512	random valid	50.3	0.333	0.375
budget 512	strict boundary	84.6	1.000	0.000
prior 0.04	symbolic proxy	15.9	0.000	1.000
prior 0.04	adversarial gate	84.6	1.000	0.000
repair	mark penalty	27.9	0.000	1.000
repair	mark + shortcut	44.6	0.000	0.000
repair	full boundary	84.6	1.000	0.000

Table 6: Compact v4 evidence table. The rows mix budget, rare-prior, and repair-ablation views of the same boundary mechanism.

A shortcut trace compiles to a permit step, a shortcut elevator step, a pickup, and a delivery step. The checker treats the shortcut as certified reachability. The executor marks contamination if PPE is absent and also checks packing. This trace demonstrates that reachability and safe reachability are different predicates.

These traces are deliberately small enough to inspect by hand. The point is not to hide behind aggregate results. A reviewer can follow each compiled action, see why the checker accepts it, and see why the hidden executor rejects it. That is the heart of a boundary audit.

APPENDIX AB: SELECTOR EQUATIONS

The language-prior selector rewards log probability and shortness. It is not a safety selector; it is included because fluent or concise plans are often used as a default heuristic. The symbolic proxy rewards checker-valid plans and adds bonuses for paperwork and shortcut completion because those features make the abstract goal easy to satisfy. The simulator proxy similarly rewards observed reachability and declaration actions. These bonuses are not arbitrary: they model a surrogate that can see a goal flag or reachable transition but cannot see hidden physical semantics.

The calibrated-boundary score subtracts a semantic uncertainty penalty from the simulator proxy. The uncertainty feature increases for paperwork completion, shortcut transitions, missing packing, and unsupported safety declarations. The adversarial gate is harsher: it assigns very low scores to checker-invalid plans, paperwork plans, shortcut plans, and unpacked plans. The uncertainty LCB subtracts uncertainty from the symbolic proxy. The strict-boundary proxy requires checker validity, explicit packing, no paperwork goal completion, and no shortcut transition.

These selectors are intentionally transparent. The paper does not claim that these are optimal learned repairs. The reason to use transparent selectors is to make the causal story testable. If a mark penalty changes the selected mode, paperwork was part of the failure. If a shortcut penalty changes the selected mode, shortcut reachability was part of the failure. If neither single-feature penalty restores execution, the boundary contract is missing more than one condition.

APPENDIX AC: EXPANDED RESULT TABLES

The high-budget budget table can be read as three regimes. At $N = 1$, all selectors are equivalent and inherit the generator mixture. At intermediate budgets, proxy selectors quickly find loopholes while repair selectors improve as robust plans appear. At $N = 512$, the separation is complete: symbolic and simulator proxies are fully collapsed, while boundary-aware repairs are fully successful in the controlled domains.

The table also clarifies why loophole rate alone is not enough. The mark-plus-shortcut penalty has loophole rate zero but execution success zero. It selects lean physical plans that still miss packing. The audit therefore tracks execution success and semantic-bad rate in addition to loophole status.

APPENDIX AD: CANDIDATE MODE NOTES

Robust grounded plans are long and explicit. They pay a proxy cost for length but satisfy execution. Redundant safe plans are even longer because they add a paperwork step after physical delivery; they still execute correctly. Lean grounded plans look plausible and do move the item, but they omit a hidden preparation requirement. They are not counted as loopholes because they do not use the symbolic completion hole, but they are semantic failures.

Paperwork loopholes are short and high scoring because they set the abstract goal through a record. Elevator loopholes and simulator lures are high scoring because they exploit reachability. Invalid plans have low proxy scores because they fail coarse preconditions. Thus the selector is not merely choosing arbitrary bad plans. It is choosing a very particular bad region: checker-valid plans whose proxy-visible completion is not execution-complete.

The candidate mode table should be treated as a diagnostic map. A real system would replace these hand-labeled modes with a failure taxonomy obtained from execution traces, simulator disagreement, human annotations, or postcondition checks. The key is to identify which modes occupy the selected proxy tail.

APPENDIX AE: FALSIFICATION CRITERIA

The paper would be falsified in this artifact if any of the following occurred. First, symbolic-proxy selection at high budget did not increase selected loophole rate. Second, simulator-proxy selection did not show the same boundary failure. Third, repair selectors failed on the same candidate pools. Fourth, candidate-level diagnostics showed that loopholes had neither proxy advantage nor hidden utility disadvantage. Fifth, strict-boundary selection failed to recover execution despite robust plans being present.

The paper would also be weakened if the only failures were checker-invalid plans. That would indicate a checker implementation bug rather than a boundary-contract bug. It would be weakened if all candidates were bad, because selection would not be the cause. It would be weakened if the repair read the hidden execution label. The tests and audit are designed to guard against these cases.

For real systems, falsification is domain specific. If high-budget proxy selection improves hidden execution monotonically, and selected examples show no boundary failures, then this audit would not diagnose a problem. That would be a good outcome. The purpose of the audit is not to force a failure; it is to make the failure measurable when it exists.

APPENDIX AF: DEPLOYMENT CHECKLIST

Before deploying a language/symbolic planner with candidate-pool selection, answer the following questions:

1. What semantic information is lost when language is compiled?
2. Which checker predicates correspond to real-world state changes?
3. Which predicates are merely records, declarations, or surrogate flags?
4. Does selected hidden utility improve with candidate budget?
5. Does selected failure taxonomy change with candidate budget?
6. Are checker pass rate and execution success reported separately?
7. Are selected examples inspected at the largest deployed budget?
8. Does a repair use only proxy-visible information?
9. Is the repair evaluated on the same candidate pools?
10. Is there a hidden holdout executor or postcondition checker?

These questions are deliberately operational. The failure studied here is not an abstract philosophical concern. It can appear in logs: a plan passes the symbolic layer, receives a high proxy score, and then

fails a hidden condition. If the system does not log enough information to answer the checklist, it is not auditable.

APPENDIX AG: EXTERNAL VALIDITY RISKS

The largest external-validity risk is that real language/planning stacks may have richer checkers than this benchmark. That is true, and richer checkers can reduce the failure. The audit does not claim otherwise. It asks whether the checker is rich enough for the deployed task. A rich PDDL model can still omit contamination, fragile handling, human authorization, timing constraints, or tool side effects.

Another risk is that real simulators may model some hidden semantics. Again, that helps. The audit should then be rerun with the simulator as the proxy and a stricter executor or postcondition suite as the hidden evaluator. The boundary always moves to the interface between the optimized proxy and the unoptimized success condition.

A third risk is that a real language model may not emit the toy loopholes. The prior sweep addresses the general version of this concern: even rare loopholes can dominate final selection if they sit in the proxy tail. A real audit would measure the actual proposal prior rather than assume it.

APPENDIX AH: DISTINCTION FROM REWARD HACKING

Reward hacking and reward-model overoptimization are close relatives, but this paper is not merely renaming them. The object here is a compiled planning boundary. The proxy is not a scalar reward learned from preferences; it is a checker or simulator attached to a symbolic abstraction. The selected failure is not arbitrary high reward; it is checker-valid hidden execution failure.

This distinction matters for repairs. Generic reward regularization may reduce overoptimization, but a planning boundary often needs explicit semantic coverage: physical handoff, packing, contamination, authorization, timing, or state persistence. The repair must ask what the symbolic layer forgot. That is why the paper uses boundary-specific diagnostics rather than only proxy regret.

APPENDIX AI: EXTENDED SELF-REVIEW

The harshest version of the review is straightforward. The benchmark is synthetic, the generator is a template sampler, and the repairs know the boundary features. The paper survives only if it stays honest about those facts. The current manuscript does so: it claims a mechanism audit, not a deployed safety result.

The next attack is that the result is too obvious. A form is not a package; of course paperwork delivery fails. The response is that the obviousness is the point. The checker abstraction made the form equivalent to delivery. The candidate-pool selector then amplified that equivalence. Real systems often fail through similarly mundane abstraction gaps.

The next attack is that the paper needs real LLMs. For a stronger empirical venue, yes. For this artifact, the controlled generator is a feature because it makes proposal prior and selected tail measurable. A real-LLM extension should reuse the audit protocol, not replace the mechanism claim with opaque examples.

The final attack is duplicate risk. The paper avoids that by making every section specific to language-to-symbolic planning: compiled actions, checker/executor divergence, physical handoff, service-elevator shortcuts, packing, and boundary gates. The result is a planning-boundary audit, not a generic sample-and-rank wrapper.

APPENDIX AJ: AUDIT LOG SCHEMA

A real deployment audit should log enough information to reconstruct the decision. For every prompt or task instance, the log should include the raw language candidate, compiled symbolic action list, checker verdict, simulator score, proxy score used for selection, selected index, execution outcome,

and postcondition failures. It should also include the candidate budget and random seed or sampling metadata. Without the full candidate pool, an investigator cannot know whether the selector chose the only plausible plan or ignored better grounded alternatives.

The log should preserve both text and symbols. Text alone hides the checker state. Symbols alone hide the language ambiguity that produced the compiled actions. The audit needs both sides of the boundary. If a plan says "file the delivery form" and the compiler emits a delivered predicate, the reviewer must see that translation.

The log should also mark whether each failure is checker-invalid, checker-valid but semantically bad, or execution-successful. These three classes lead to different engineering fixes. Checker-invalid failures call for better generation or precondition filtering. Checker-valid semantic failures call for boundary modeling. Execution successes can still be used to calibrate how much proxy score is sacrificed by repairs.

APPENDIX AK: ACCEPTANCE-GRADE EXPERIMENT DESIGN

An acceptance-grade real-system extension would use the same experimental shape but replace synthetic templates with a real model and task suite. The candidate generator should be a deployed or proposed language planner. The compiler should be the actual interface used by the system: PDDL, tool calls, robot actions, code, or simulator commands. The checker should be the actual symbolic validator. The hidden executor should be a separate evaluation source such as physical trials, high-fidelity simulation, hidden tests, or audited postconditions.

The experiment should be compute matched. Every selector must receive the same candidate pools. A verifier or repair cannot be allowed to sample more candidates unless the baseline receives the same budget. Otherwise the result confounds generation with selection. The experiment should also include a random valid baseline, because that baseline reveals the quality of the checker-valid pool before proxy optimization.

The paper should report at least three budgets below the deployed budget and one budget above it. If the deployed system uses 64 candidates, report 1, 8, 32, 64, and 128. If the deployed system uses 512 candidates, report a curve up to 512. The failure is an inference-interface phenomenon, so a single budget does not show whether selection pressure is causing the outcome.

APPENDIX AL: BOUNDARY FEATURE DISCOVERY

In the toy domains the boundary features are known: paperwork, shortcut transitions, missing packing, and unsupported declarations. In real systems they may not be known. A practical audit can discover them by comparing the highest-proxy failed candidates with successful candidates that have lower proxy scores. The comparison should ask which compiled actions, textual phrases, simulator states, or checker predicates are enriched in failures.

Feature discovery should be done on a development split. The final evaluation must then test the proposed boundary features on heldout tasks or heldout instances. Otherwise the repair may simply overfit the observed failures. The toy artifact avoids this issue by making the features part of the controlled mechanism and by not claiming a universal learned detector.

One useful real-system tool is counterfactual compilation. Take a selected failed plan and change the language step that triggers a suspicious compiled action. If the checker still passes but the hidden executor changes, the boundary feature is likely meaningful. Another tool is action masking: remove one class of compiled action from the candidate pool and measure whether hidden execution improves.

APPENDIX AM: REVIEWER SCORE SIMULATION

A harsh reviewer might score novelty as moderate rather than high because the general idea of proxy overoptimization is well known. The intended novelty is not that proxies can be wrong. It is the specific boundary audit: language plans are compiled into symbolic actions, checker validity diverges

from hidden execution, and candidate-pool selection concentrates checker-valid semantic failures. The paper should be evaluated as a mechanism artifact.

The same reviewer might score empirical strength as limited because there are no real model checkpoints. That criticism is fair. The v4 paper compensates by making the controlled artifact unusually complete: high-budget stress, rare-prior stress, repair ablations, candidate calibration, failure cases, claim audits, and a reproducibility path. The right score depends on venue expectations. For a workshop or diagnostic track, the artifact is strong. For a full empirical systems track, it needs real-model extension.

Clarity should be high if the paper keeps its scope. The title names planning, the abstract states the synthetic boundary, and the appendices enumerate unsupported claims. The largest clarity risk is overusing generic candidate-pool language. The manuscript mitigates that by repeatedly grounding the discussion in compiled actions, checkers, executors, and boundary semantics.

APPENDIX AN: ADDITIONAL NEGATIVE EXAMPLES

Not every failed plan is a loophole. A missing-key plan fails the checker and therefore does not demonstrate the boundary problem. An unmoved plan that tries to deliver without pickup also fails coarse preconditions. These failures are useful as controls because the proxy selectors assign them low scores. If the selected high-budget failures were mostly checker-invalid, the paper would be about checker weakness or generator quality, not boundary exploitation.

Lean grounded plans are more subtle. They may physically move the item but miss packing, carrier, or preparation. They are checker-valid and semantically bad, but they are not counted as loopholes because they do not use paperwork or shortcut completion. The mark-plus-shortcut repair selects these plans, which is why it has zero loophole rate and zero execution success. This is a useful negative example for single-feature repairs.

These distinctions make the evaluation harder to game. A repair cannot claim success by only reducing one failure label. It must recover execution success. The paper therefore reports utility, success, semantic-bad rate, and loophole rate together.

APPENDIX AO: FINAL EVIDENCE CHAIN

The evidence chain is: a boundary contract can omit hidden execution semantics; the toy domains instantiate that omission; the finite-pool law predicts amplification when loopholes have higher proxy scores; the budget experiment shows amplification; the rare-prior sweep shows the effect persists when loopholes are uncommon; the repair ablation shows which boundary features matter; candidate calibration shows that proxy score is high exactly where hidden utility is low; and the claim audit verifies that the final PDF and results correspond to this v4 artifact.

This chain is intentionally redundant. Each link answers a different attack. The proof answers whether the mechanism is coherent. The budget curve answers whether it appears in the implementation. The prior sweep answers whether it depends on a high loophole rate. The ablation answers whether repairs are mechanistically tied to boundary features. The calibration answers whether the proxy is merely noisy or specifically wrong in the selected tail. The audit answers whether the delivered paper is stale.

APPENDIX AP: REAL-WORLD ANALOGUE CATALOGUE

The toy tasks correspond to common abstraction patterns. Paperwork delivery is an analogue of systems that treat database state as physical state: a ticket is closed, an order is marked shipped, a medication is marked administered, or a file is marked reviewed. In each case, the symbolic predicate may be true in the information system while the physical or human process is incomplete.

Shortcut reachability is an analogue of route planners and tool agents that optimize reachability without modeling side effects. A robot can reach a room through a route that violates sterility. A software agent can reach a state by using an unsafe API shortcut. A logistics system can satisfy

a route predicate while violating temperature, custody, or authorization constraints. The checker reachability predicate is not wrong; it is incomplete.

Simulator lures are analogues of learned or approximate simulators that model the visible transition but not the hidden cost. If a simulator rewards a state flag, a plan can learn to set that flag without satisfying the underlying condition. The audit is therefore useful wherever simulators are used as selection proxies rather than as complete execution environments.

These analogues are not empirical claims. They are guidance for where to apply the audit. A real paper should choose one domain, define its boundary contract, log candidate pools, and measure hidden execution directly. The toy artifact supplies the experimental grammar.

APPENDIX AQ: ETHICS AND MISUSE

The work is defensive. It shows how a planning stack can overtrust a symbolic boundary, and it provides diagnostics for detecting that overtrust. The main misuse risk is rhetorical rather than operational: someone could cite the toy result as proof that symbolic planning, verification, or simulators are generally unsafe. The paper does not support that claim. It supports the narrower claim that incomplete boundary semantics can be amplified by proxy selection.

Another risk is false reassurance. A system could add a superficial boundary penalty, observe improvement on known failures, and deploy without hidden execution evaluation. The repair ablations are included partly to prevent that mistake. Single-feature repairs can remove one visible failure and still miss another hidden condition. The correct deployment lesson is to maintain a holdout executor or postcondition suite.

The benchmark uses no human subjects, external APIs, private data, or real-world robotic actions. It is safe to run locally. The ethical value comes from making an otherwise subtle interface failure visible before it appears in a higher-stakes system.

APPENDIX AR: FIFTY-POINT SELF-ATTACK

1. Is the title specific to planning boundaries? Yes.
2. Does the abstract state synthetic scope? Yes.
3. Does the paper avoid universal safety claims? Yes.
4. Does it include a finite-pool mechanism? Yes.
5. Does it test more than one task? Yes.
6. Does it test more than one loophole type? Yes.
7. Does it include $N=512$ stress? Yes.
8. Does it test rare loophole priors? Yes.
9. Does it compare symbolic and simulator proxies? Yes.
10. Does it include random valid selection? Yes.
11. Does it include repair ablations? Yes.
12. Does it explain why single-feature repairs fail? Yes.
13. Does it include candidate-level calibration? Yes.
14. Does it include checker/executor gap plots? Yes.
15. Does it include failure cases? Yes.
16. Does it include exact action traces? Yes.
17. Does it explain compiler loss? Yes.
18. Does it explain hidden executor state? Yes.
19. Does it separate semantic-bad from loophole? Yes.
20. Does it preserve a narrow claim boundary? Yes.
21. Does it update build output to v4? Yes.

-
- 1242 22. Does the audit require 25 pages? Yes.
1243 23. Does the audit require expansion claims? Yes.
1244 24. Does the repo include generated CSVs? Yes.
1245 25. Does the repo include generated figures? Yes.
1246 26. Does the paper cite adjacent agent and planning work? Yes.
1247 27. Does it distinguish reward hacking? Yes.
1248 28. Does it state external-validity risks? Yes.
1249 29. Does it provide a real-system porting protocol? Yes.
1250 30. Does it include deployment checklist questions? Yes.
1251 31. Does it avoid stale v2 filenames in the paper? Yes.
1252 32. Does it avoid unrelated paper names? Yes.
1253 33. Does it keep CPU use reasonable? Yes.
1254 34. Does it avoid claiming repairs are universal? Yes.
1255 35. Does it explain why larger N can help aligned repairs? Yes.
1256 36. Does it identify selected-tail calibration as the issue? Yes.
1257 37. Does it include negative controls? Yes.
1258 38. Does it include falsification criteria? Yes.
1259 39. Does it include ethics and misuse notes? Yes.
1260 40. Does it make the source folder recoverable? Yes.
1261 41. Does it produce a Desktop PDF? Yes.
1262 42. Does it produce a repo final PDF? Yes.
1263 43. Does it keep the branch pushable? Yes.
1264 44. Does it explain why checker pass is insufficient? Yes.
1265 45. Does it explain why random valid is not enough? Yes.
1266 46. Does it explain why mode accounting matters? Yes.
1267 47. Does it explain how to find real boundary features? Yes.
1268 48. Does it discuss holdout execution? Yes.
1269 49. Does it remain falsifiable? Yes.
1270 50. Does it read as a distinct paper beside the others? Yes.
1271
1272
1273
1274
1275
1276
1277
1278
1279

1280 APPENDIX AS: ARTIFACT GOVERNANCE

1281
1282 The repository treats the paper, code, generated results, figures, claim audit, and Desktop filename
1283 as one artifact. This matters because a paper can be scientifically improved while the visible PDF
1284 remains stale, or a result file can be regenerated while the manuscript still reports old numbers. The
1285 v4 workflow therefore makes the build script write both a repository final PDF and a Desktop final
1286 PDF with the same v4 name. The claim audit then checks the expansion claim file and the page count
1287 of both PDFs.

1288 The artifact governance rule is simple: a result is not final until it is recoverable. A fresh agent should
1289 be able to start from the Desktop PDF name, find the source folder through the source map, inspect
1290 the GitHub repository, run the claim audit, and rebuild the same paper. This is why the repository
1291 stores generated CSVs and figures rather than only prose. It is also why the paper avoids relying on
1292 unstated local notebooks or hidden manual analysis.

1293 The governance layer is not bureaucracy. It is part of scientific rigor for a collection of related papers.
1294 When several projects study candidate-pool selection, stale filenames and shared templates create
1295 review risk. The artifact must make its identity obvious: this repository is the language-to-symbolic
planning boundary audit, and its final output is the v4 submission PDF.

APPENDIX AT: MINIMAL REVIEWER REPRODUCTION

A reviewer can reproduce the central claim without reading every file. First, run the tests to verify that the compiler, checker, executor, selector, and expansion suite behave deterministically. Second, run the v4 expansion suite. Third, open the expansion claim file and confirm that all six checks are true. Fourth, inspect the budget rows at $N = 512$: symbolic and simulator proxy selection should have utility 15.9, success 0.0, and loophole rate 1.0. Fifth, inspect the repair rows: adversarial gate, uncertainty LCB, strict-boundary, and full-boundary repairs should recover success 1.0. Sixth, build the PDF and run the claim audit.

This minimal path tests the whole chain. The tests cover local mechanics. The expansion suite regenerates evidence. The claim file checks the paper’s central conditions. The summary rows expose exact numbers. The build step verifies that the manuscript compiles from source. The claim audit verifies that the final paper is not a stale short version.

The reviewer can then inspect examples if desired. The failure-case CSV shows selected high-budget proxy failures. The candidate diagnostics file shows why the proxy tail is wrong: paperwork and shortcut plans have high proxy scores and low hidden utility. The figures summarize the same evidence visually. This is enough to accept or reject the narrow mechanism claim without trusting any unstated analysis.

APPENDIX AU: BOUNDARY-CONTRACT PORTING TEMPLATE

The audit is easiest to transfer when the reviewer writes the boundary contract before running candidate selection. The contract should name what the language model is allowed to propose, what the compiler preserves, what the checker can observe, what the simulator approximates, and what the hidden executor will penalize. A missing entry in this contract is a scientific object, not a software inconvenience: it is exactly where a plan can pass the proxy while failing the world.

Contract field	Reviewer-facing question
Language action space	Which phrases or program templates can create a candidate action, and which of them are merely fluent aliases rather than grounded operations?
Compiler semantics	Which preconditions, effects, resource constraints, ownership constraints, and temporal constraints survive translation into the symbolic representation?
Checker visibility	Which success predicates can be satisfied without executing the intended operation, and which hidden conditions are never read by the checker?
Simulator fidelity	Which latent costs are approximated or omitted by the simulator score, especially after a candidate has been selected from a large pool?
Executor oracle	What postconditions would a skeptical deployment monitor measure if it did not trust the checker or simulator?
Selection pressure	What happens to the selected tail as N increases, and does the tail contain modes that were rare in the proposal distribution?

Table 7: A concrete template for porting the audit to another language-to-symbolic planner. The key requirement is to write the hidden executor oracle before interpreting larger candidate pools as better planning.

This template also separates engineering fixes from scientific claims. Adding a stricter checker is an engineering fix only after the executor oracle has shown which hidden conditions matter. Penalizing one known shortcut is not equivalent to repairing the boundary; the v4 ablations show that single-feature repairs can leave success at zero. Conversely, a repair that looks conservative in the symbolic layer can be beneficial if it removes selected-tail loopholes while preserving grounded plans. The paper therefore reports both the proxy score and the hidden execution score for the selected candidate, not only aggregate candidate quality.

For a real robotics or tool-use system, the porting procedure would have five steps. First, log the raw candidate pool before filtering, because the selected candidate alone hides whether the system was robust or merely lucky. Second, tag each candidate with proxy score, simulator score, checker result, and executor result. Third, stratify candidates by failure mode rather than by success alone; loopholes, invalid plans, redundant safe plans, and grounded plans should be visible as separate modes. Fourth,

rerun selection under the actual inference budget used by the system. Fifth, evaluate repairs on a held out set of boundary failures whose features were not used to design the repair.

The last step is where many optimistic audits become weak. If the same failure examples are used to invent and evaluate a boundary penalty, the experiment only proves that those examples can be patched. A submission-ready audit should reserve either tasks, templates, object attributes, or executor predicates for holdout evaluation. The synthetic artifact uses deterministic templates, so it does not claim this form of external generalization. Instead, it makes the weaker and fully reproducible claim that the mechanism is present, that it is amplified by selection pressure, and that repairs must be judged by hidden execution rather than by proxy improvement alone.

The template is deliberately procedural because boundary failures are often invisible in high-level prose. A paper can say that it uses symbolic validation, simulation, and execution feedback while leaving unspecified which facts cross each interface. A reviewer should be able to annotate a system diagram with the six contract fields above. If any field is unknown, the paper has not made the boundary auditable. In that case, increasing N , adding self-consistency, or sampling more fluent plans is not evidence of safer planning. It is more pressure on an unmeasured interface.

REFERENCES

- Liu et al. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. arXiv, 2023.
- Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.
- Ahn et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. CoRL, 2022.
- Huang et al. Inner Monologue: Embodied Reasoning through Planning with Language Models. CoRL, 2022.
- Yao et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS, 2023.
- Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023.
- Liang et al. Code as Policies: Language Model Programs for Embodied Control. ICRA, 2023.
- Gao et al. Scaling Laws for Reward Model Overoptimization. ICML, 2023.
- Everitt et al. Reward Tampering Problems and Solutions in Reinforcement Learning. Synthese, 2021.
- Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023.
- Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. TMLR, 2023.
- Silver and Chitnis. PDDL Gym: Gym Environments from PDDL Problems. ICAPS Workshop, 2020.
- Towers et al. Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv, 2024.